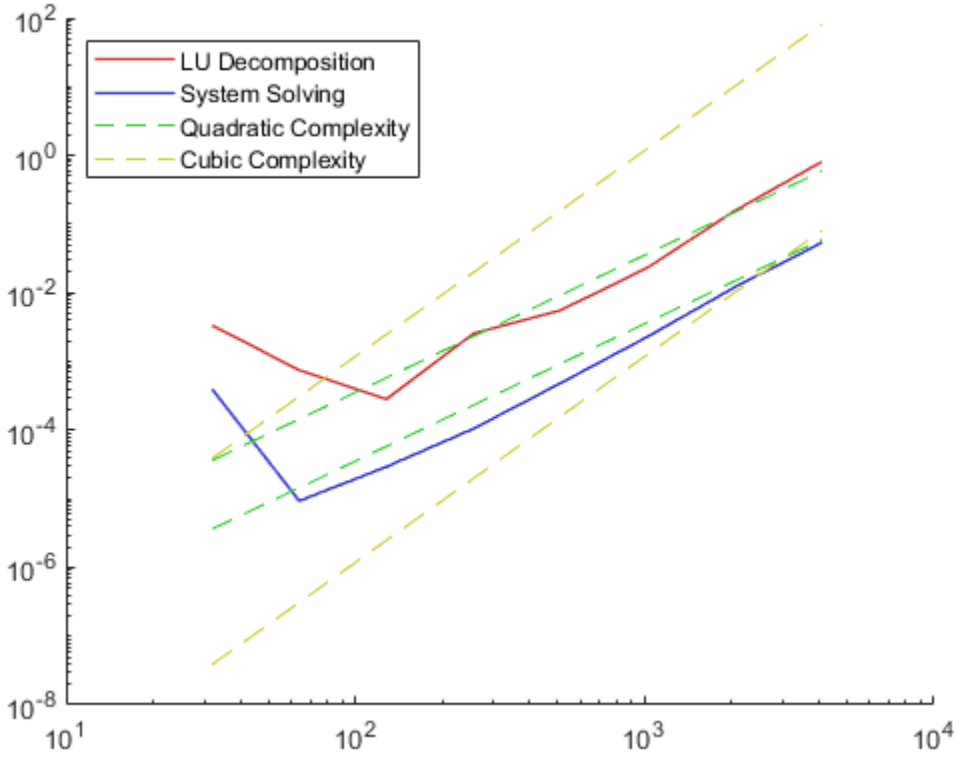
```
ns = 2.^(5:12);
ntrials = 5;
ts1 = zeros(size(ns));
for i = 1:length(ns)
    n = ns(i);
    fprintf('running mat-mat test %d of %d, n = %d\n',i,length(ns),n);
    start = tic;
    for j = 1:ntrials
        A = randn(n,n);
        [L,U,P] = lu(A);
    end
    ts1(i) = toc(start)/ntrials;
end

ts2 = zeros(size(ns));
for i = 1:length(ns)
    n = ns(i);
    fprintf('running mat-mat test %d of %d, n = %d\n',i,length(ns),n);
    for j = 1:ntrials
        A = randn(n,n);
        [L,U,P] = lu(A);
        b = randn(n,1);
        start = tic;
        x = U\(L\(P*b));
        ts2(i) = toc(start)/ntrials + ts2(i);
    end
end

axes('XScale', 'log', 'YScale', 'log')
hold on
plot(ns,ts1,DisplayName="LU Decomposition", Color=[0.8 0.2 0.2],LineWidth=1)
plot(ns,ts2,DisplayName="System Solving", Color=[0.2 0.2 0.8],LineWidth=1)
plot(ns,3000*(ns.^2*ts1(end)/ns(end)^3),DisplayName="Quadratic
Complexity",LineStyle="--",Color=[0.2 0.8 0.2])
plot(ns,100*(ns.^3*ts1(end)/ns(end)^3),DisplayName="Cubic
Complexity",LineStyle="--", Color=[0.8 0.8 0.2])
legend('Location','northwest','AutoUpdate','off')
plot(ns,300*(ns.^2*ts1(end)/ns(end)^3),LineStyle="--",Color=[0.2 0.8 0.2])
plot(ns,0.1*(ns.^3*ts1(end)/ns(end)^3),LineStyle="--", Color=[0.8 0.8 0.2])
hold off
```

```
running mat-mat test 1 of 8, n = 32
running mat-mat test 2 of 8, n = 64
running mat-mat test 3 of 8, n = 128
running mat-mat test 4 of 8, n = 256
running mat-mat test 5 of 8, n = 512
running mat-mat test 6 of 8, n = 1024
running mat-mat test 7 of 8, n = 2048
running mat-mat test 8 of 8, n = 4096
running mat-mat test 1 of 8, n = 32
running mat-mat test 2 of 8, n = 64
running mat-mat test 3 of 8, n = 128
running mat-mat test 4 of 8, n = 256
```

```
running mat-mat test 5 of 8, n = 512
running mat-mat test 6 of 8, n = 1024
running mat-mat test 7 of 8, n = 2048
running mat-mat test 8 of 8, n = 4096
```



Published with MATLAB® R2023b